Week4

关于design pattern，可以参考此文章设计模式（Design Patterns） – 黄文博 – 博客园 (cnblogs.com)

基本包含了所有内容

The Decorator Design pattern 装饰设计模式

The Decorator design pattern involves

- A class which implements an interface 实现接口的类
- Inside it a variable of the same interface type whose value is set in the constructor 在其内部是一个具有相同接口类型的变量，其值在构造函数中设置
- Each method from the interface has code which calls the same method with the same arguments on that variable, plus some extra work 接口中的每个方法都有调用相同方法的代码，并在该变量上使用相同的参数，再加上一些**额外的工作**

举个例子：

```
class QuackCounter implements Quackable {
  private Quackable myQuacker;
  private int count;
  public QuackCounter(Quackable q) {
    myQuacker=q;
    //在其内部是一个具有相同接口类型的变量，其值在构造函数中设置
  }
  public int getCount() {
    return count;
  }
  public void quack()
  {
    count++;//一些额外的工作
    myQuacker.quack();
  }
}
//以下是实现计数的代码
Quackable q1= new Duck("Donald");
Quackable q2= new QuackCounter(q1);
Value v = t.process(…,q2,…);
int quackcount = q2.getCount()
```

- In this example, the extra work is to increase the value of the count by one 在本例中，额外的工作是将计数的值增加1

- The effect is to “decorate” the methods of an object of the interface type with the code that does the extra work 其效果是用执行额外工作的代码“修饰”接口类型对象的方法

Wrapper design patterns 包装器设计模式

The Decorator design pattern is an example of a general sort of pattern called a wrapper pattern

装饰设计模式是称为包装器模式的一般模式的一个示例

A wrapper pattern is any pattern where an object (called the inner object) is referred to by a variable inside another object (the outer object), and each method call on the outer object results in a method call on the inner object

包装器模式是由另一个对象(外部对象)内部的变量引用对象(称为内部对象)的任何模式，并且对外部对象的每个方法调用都会导致对内部对象的方法调用

- 相当于包装了一下，用外面的就相当于用里面的，可以保护里面的

In our example, QuackDecorator was the outer object wrapping an inner object which could be any object of type Quackable

在我们的例子中，QuackDecorator是包裹内部对象的外部对象，内部对象可以是任何Quackable类型的对象

In the Decorator pattern, the inner object and outer object must both be of a class which implements the same interface, in this case Quackable

在Decorator模式中，内部对象和外部对象必须都属于实现相同接口的类，在本例中为Quackable

Another wrapper pattern is the Adapter pattern, which involves an outer object which implements an interface wrapping an inner object which is of a class that does not implement the interface

另一种包装器模式是适配器模式，它涉及到一个实现接口的外部对象包装一个不实现接口的类的内部对象

The Adapter design pattern 适配器设计模式

The Adapter design pattern “adapts” an object of one class to fit into an interface when the object is not of a type which implements the interface

当对象不属于实现接口的类型时，适配器设计模式“调整”一个类的对象以适应接口

It works by “wrapping” the object as an inner object in an outer object which is of a class that does implement the interface

它的工作原理是将对象作为内部对象“包装”在实现接口的类的外部对象中

Each method in the interface has to be implemented by code which performs what is regarded as the “equivalent” in the class of the inner object

接口中的每个方法都必须由代码实现，这些代码在内部对象的类中执行被视为“等效”的操作

举个例子来看：

```
class Goose {
  private String myName;
  public Goose(String theName) {
    myName=theName;
  }
```

```
  public void honk() {
    System.out.println(myName+": Honk! ");
  }
}
```

- 目的是让这里的honk()和上面的quack()等效

```
class GooseAdapter implements Quackable {
  private Goose myGoose;
  public GooseAdapter(Goose aGoose) {
    myGoose=aGoose;
  }
  public void quack() {
    myGoose.honk();
  }
}
…
interface Quackable {
  public void quack();
}
```

- 这里的quack()看上去和上面的QuackCounter类差不多

```
Goose g = new Goose("Gertrude");
Quackable q3 = new GooseAdapter(g);
Value v = t.process(…,q3,…);
```

- 这样我们和上面的例子一样，用的都是Quackable的对象，很好的包装了实际的类（Duck/Goose）

Why not just change the code?

Other code may already make use of class Duck, class Goose, or method process, we cannot change the code for them without having to check whether the change causes that code to stop working properly (OCP – Open Closed Principle)

其他代码可能已经使用了类Duck、类Goose或方法process，我们不能在不检查更改是否导致代码停止正常工作(OCP)的情况下为它们更改代码。

If we added code to count the number of calls to the method quack made in the call to the method process, we are making that method do two separate tasks. The QuackCounterwrapper separates out those two separate tasks (SRP – Single Responsibility Principle)

如果我们添加代码来计算在方法进程调用中对方法quack的调用次数，我们就使该方法执行两个独立的任务。QuackCounterwrapper将这两个独立的任务(SRP)分离出来。

If we wanted to count the number of calls to the method quackmade in a call to another method which takes an argument of type Quackable, we can just use the wrapper we already have (DRY – Don't Repeat Yourself)

如果我们想要在调用另一个方法(该方法接受类型为Quackable的参数)时计算对方法quackmade的调用次数，我们可以使用我们已有的包装器(DRY)。

We may not want to pass full access to Goose objects to other code, passing the GooseAdapter object passes only the abilityto call the method honk on a Goose object (DIP – The Dependency–Inversion Principle)

我们可能不希望将对Goose对象的完全访问权传递给其他代码，传递GooseAdapter对象只传递在Goose对象(DIP)上调用方法honk的能力。

Another sort of Adapter (Adapter using Inheritance)

```
class QuackingGoose extends Goose implements Quackable{
  public QuackingGoose(String name) {
    super(name);
  }
  public void quack() {
    honk();
  }
}
```

- This is another sort of Adapter, this time using inheritance rather than delegation. Here a QuackingGoose object IS–A Goose object, whereas previously a GooseAdapter HAS–A Goose object 这是另一种适配器，这次使用继承而不是委托。这里的QuackingGoose对象 **IS–A** Goose对象，而之前的GooseAdapter **HAS–A** Goose对象
- Passing a QuackingGoose object to other code passes access to all Goose methods, whereas passing a GooseAdapter object only passes access to honk called indirectly through quack. 将QuackingGoose对象传递给其他代码会传递对所有Goose方法的访问权，而传递GooseAdapter对象只会传递对通过quack间接调用的honk的访问权。

Design patterns

Design patterns solve a particular problem, here:

- How can we add an additional task to an existing method without changing the code of that method? 如何在不更改现有方法代码的情况下向该方法添加额外的任务?
- How can we fit a class into an interface which it is not declared as implementing without changing the code of that class? 在不改变类的代码的情况下，我们如何将类放入未声明为实现的接口中?

Design patterns are general, we can re–use the same pattern in many different circumstances

设计模式是通用的，我们可以在许多不同的情况下重用相同的模式

Design patterns go beyond what is provided directly in the programming language

设计模式超越了编程语言直接提供的内容

Design patterns often involve a use of code which a novice programmer probably would not think of

设计模式通常涉及到新手程序员可能想不到的代码使用

Design patterns provide us with a “vocabulary”, here we can talk to other programmers about using a

“decorator” or an “adapter” and they will know what we mean

设计模式为我们提供了一个“词汇表”，在这里我们可以和其他程序员讨论使用“装饰器”或“适配器”，他们会明白我们的意思

The Composite Design Pattern 复合设计模式（可以放多个对象[利用ArrayList]的wrapper pattern）

Another example of a wrapper pattern, this time a **collection** of objects of a type is wrapped and given the behaviour of a single object of that type:

另一个包装器模式的例子，这一次，一个类型的对象集合被包装，并被赋予该类型单个对象的行为:

```
class QuackComposite implements Quackable {
  private List quackers;
  private int count;
  public QuackComposite() {
    quackers = new ArrayList();
  }
  public void addQuacker(Quackable q) {
    quackers.add(q);
  }
  public void quack() {
    for(Quackable aQuacker : quackers)
      aQuacker.quack();//所有ArrayList里的对象都执行
  }
}
```

Immutable View

We have seen previously how problems can be caused by aliasing: two different variables that refer to the same object,

我们之前已经看到了混叠是如何引起问题的:两个不同的变量引用同一个对象,

A method call made on one of the variables that causes a change of state to the object it refers to means the object the other variable refers to also changes state as it is the same object

对其中一个变量进行的方法调用导致它所引用的对象的状态发生变化，这意味着另一个变量所引用的对象也会改变状态，因为它是同一个对象

For example, exposing the representation was when a reference was passed that meant an internal variable of an object was aliased by another variable outside that object

例如，当传递的引用意味着对象的内部变量被该对象外部的另一个变量别名时，就会暴露表示

This problem can be avoided by passing an “immutable view” of an object rather than a direct reference

这个问题可以通过传递对象的“不可变视图”而不是直接引用来避免

This is another example of the general principle of a wrapper design pattern. It involves calling the same method on the inner object if it does not make a change, but for methods that do make changes throwing an exception

instead

这是包装器设计模式一般原则的另一个示例。它涉及到在内部对象上调用相同的方法，如果它没有进行更改，但是对于进行更改的方法则抛出异常

```
class ImmutableRectangle implements Rectangle {
  private Rectangle myRectangle;
  public ImmutableRectangle(Rectangle r) {
    myRectangle=r;
  }
  public int getWidth() {
    return myRectangle.getWidth();
  }
  public int getHeight() {
    return myRectangle.getHeight();
  }
  public int area() {
    return myRectangle.area();
  }
  //不让你更改，所以所有setter都是抛出异常
  public void setWidth(int w) {
    throw new UnsupportedOperationException();
  }
  public void setHeight(int h) {
    throw new UnsupportedOperationException();
  }
}
```

阶段总结

- wrapper pattern (wp)包括decorator pattern (dp)和adapter pattern (ap)，这个是主要的两大类，二者的不同之处在于：dp的构造器中，放的是和接口相同的类（quackable），ap的构造器里放的是和接口不同的类（goose）。但二者都要被包装成相同的类在main函数中被调用（quackable）
- ap还可以利用继承
- wp可以利用ArrayList进行复合，来一次处理多个对象（复合的类似于dp）
- wp的另一种可以直接利用抛出异常来杜绝改变的可能（Immutable view）

The Observer Pattern 观察者模式

The Observer design pattern is intended to cover cases where there is a link between two classes such that an action on an object of one of the classes is “observed” by objects of the other class

观察者设计模式旨在涵盖两个类之间存在链接的情况，以便其中一个类的对象上的操作被另一个类的对象“观察”到

Its most common use is to connect objects which store data or perform actions to other objects which deal with “input/output”

它最常见的用途是将存储数据或执行操作的对象**连接到处理“输入/输出”的其他对象**。

Here “input/output” could mean writing data to a file or database, or displaying on a graphical user interface

这里的“输入/输出”可能意味着将数据写入文件或数据库，或者在图形用户界面上显示

Model–View–Controller

It is generally agreed that one of the most basic divisions of responsibility in a software system is between those aspects which interact with a human user – the “view” – and those aspects which deal with the details of what is happening internally – the “model”. The “controller” is the link between these two parts

人们普遍认为，软件系统中最基本的职责划分之一是与人类用户交互的方面(“视图”)和处理内部发生的细节的方面(“模型”)。“控制器”是这两个部分之间的链接

For example, in a computer game, one part of the system will hold the details of the pieces and characters of the game, and make them change according to the rules of the game, another part will produce the screen picture that the human player of the game sees

例如，在电脑游戏中，系统的一部分将保存游戏中的棋子和角色的细节，并使它们根据游戏规则变化，另一部分将产生游戏玩家所看到的屏幕图像

If a single object held the internal details of characters in the game, and also dealt with displaying their representation on the screen, that would be breaking the Single Responsibility Principle

如果单个对象包含游戏中角色的内部细节，并且还处理在屏幕上显示角色的表现，那么这就违反了单一责任原则

It would lead to code that is hard to understand and change

这将导致代码难以理解和更改

It would make it difficult to change the screen appearance of the game as the code that produces the screen appearance is mixed up with the code that deals with the other aspects of the game

因为产生屏幕外观的代码与处理游戏其他方面的代码混合在一起，所以改变游戏的屏幕外观将变得非常困难

Java中的observer

Java的API中已经为我们提供了Observer模式的实现。具体由java.util.Observable类和java.util.Observer接口完成。

前者有两个重要的方法：

- setChanged：设置内部状态为已改变
- notifyObservers(Object obj)：通知观察者所发生的改变，参数obj是一些改变的信息.

【以及

```
public void addObserver(Observer o)
```

```
public void deleteObserver(Observer o)
```

这两个可以添加删除的方法】

后者有一个核心方法：

- update(Object obj)：相应被观察者的改变，其中obj就是被观察者传递过来的信息，**该方法会在notifyObservers**

被调用时自动调用。

下面是Observer模式的实现过程：

- 创建一个被观察者，继承java.util.Observable
- 创建一个观察者，实现java.util.Observer接口
- 注册观察着，调用addObserver(Observer observer)
- 在被观察者改变对象内部状态的地方，调用setChanged()方法，然后调notifyObservers(Object)方法，通知被观察者
- 在观察者的update(Object)方法中，对改变做出响应。

来源：https://blog.csdn.net/weixin_36311421/article/details/114462257

举个例子：

- 被观察者（继承Observable这个抽象类）

```
class DogBot extends Observable{
  protected int hungry,tired;
  private String name;
  public DogBot(String nm, int h, int t) {
    name=nm;
    hungry=h;
    tired=t;
  }
  public boolean eat() {
    if(hungry>6) {
      hungry-=3;
      setChanged();
      notifyObservers("eat");
      return true;
    }
    else
      return false;
  }
  public void rest() {
    hungry++;
    tired-=2;
    setChanged();
    notifyObservers("rest");
  }
```

```
  public void play() {
    hungry+=2;
    tired+=3;
    setChanged();
    notifyObservers("play");
  }
  public String getName() {
    return name;
  }
  public String noise() {
    if(hungry>8 && tired<11)
      return name+": whine ";
    else if(tired>7 && tired>hungry)
      return name+": snore ";
    else
      return name+": woof ";
  }
}
//观察代码，我们可以看到，每一个hungry和tired的变动，下面都还有setChanged方法和notifyObservers方法
```

- 接下来是观察者，implements Observer这个接口

```
class DogWatcher implements Observer {
  private String name;
  DogWatcher(String nm) {
    name=nm;
  }
  //每次notifyObservers都调用update
  public void update(Observable obj, Object action) {
    if(obj instanceof DogBot) {
      DogBot dog = (DogBot) obj;
      System.out.print("** "+name+" observes ");
      System.out.println(dog.getName()+" "+action);
    }
  }
  public String toString() {
    return "Dog Watcher: "+name;
```

```
}
```

- main里的代码实现过程

```
DogBot patch = new DogBot("Patch",4,2);
DogBot rover = new DogBot("Rover",9,6);
DogWatcher jim = new DogWatcher("Jim");
DogWatcher fred = new DogWatcher("Fred");
rover.addObserver(fred);  //fred是rover的observer
patch.addObserver(jim);  //jim是patch的observer
System.out.println("Patch plays");
patch.play();  //调用了jim的update
System.out.println("Rover plays");
rover.play();  //调用了fred的update
patch.addObserver(fred);
patch.play();  //调用了fred的update
rover.deleteObserver(fred);
rover.play();  //调用了jim的update
if(patch.eat())
System.out.println(" Patch eats");
else
System.out.println(" Patch decides not to eat");
```

Notes on Java's Observer code

We do not have to write any code to maintain the list of observers, it is inherited from Observable

我们不需要编写任何代码来维护观察者列表，它是从Observable继承来的

It is flexible, an Observable object can have any number of Observer objects, an Observer object can observe any number of Observable objects

它很灵活，一个可观察对象可以有任意数量的观察者对象，一个观察者对象可以观察任意数量的可观察对象

The code for the Observable object passes whatever information it likes to the Observer objects ("push")

Observable对象的代码将它喜欢的任何信息传递给Observer对象(" push ")

The code for the Observer object can get additional information by calling public methods on the Observable objects ("pull")

Observer对象的代码可以通过调用Observable对象的公共方法来获取额外的信息(" pull ")

The code which extends Observable cannot access the Observer objects directly, an object of a class which extends Observable does not "know" what its observers are

继承Observable的代码不能直接访问Observer对象，继承了Observable的类的对象并不"知道"它的观察者是什么

DogBot例子与OCP（Open-Closed Principle）

We have not followed OCP here because we have modified the class DogBot in order to make DogBot objects observable

这里我们没有遵循OCP，因为我们已经修改了类DogBot以使DogBot对象可观察

Also, we could not make DogBot extend Observable if it already extended another class, because classes can only extend one other class

此外，如果DogBot已经扩展了另一个类，我们就不能让它扩展Observable，因为类只能扩展另一个类

We can get round this by using the Decorator pattern

我们可以通过使用Decorator模式来解决这个问题

```
class ObservableDogBot
extends Observable
{
 private DogBot myDogBot;

 public ObservableDogBot(DogBot d)
 {
  myDogBot = d;
 }

 public boolean eat()
 {
  if(myDogBot.eat()) {
      setChanged();
      notifyObservers("eat");
      return true;
  }
  else
     return false;
 }

// continued …
```

```
public void rest() {
  myDogBot.rest();
  setChanged();
  notifyObservers("rest");
 }

 public void play() {
  myDogBot.play();
  setChanged();
  notifyObservers("play");
 }

 public String getName() {
  return myDogBot.getName();
 }

 public String noise() {
  return myDogBot.noise();
 }
}
```

也可以把dogbot写成接口

- We should have made `DogBot` an interface type:

```
interface Dogbot {
   public boolean eat();
   public void rest();
   public void play();
   public String noise();
   public String getName();
}
```

- with `class PlainDogBot implements DogBot`
  (code otherwise as for `DogBot`, but using name `PlainDogBot`)
- Then we could have

```
class ObservableDogBot
extends Observable implements DogBot
```

  and no other changes to code except in constructor calls

- 这样我们就做到了model和view的分离，具体代码看PPT Topic16 P28起，和上面的原理基本相同

Factory Methods 工厂方法

```
class PlainDogBot implements DogBot
```

```
class GreedyDogBot extends PlainDogBot
```

比如说，我们在一个方法里要创建这两个其中一个，我们就可以利用工厂方法（因为可以return，而constructor不可以返回他俩任何一个）

```
public static DogBot makeDogBot(String nm, int h, int t, boolean greedy) {
  if(greedy)
    return new GreedyDogBot(nm,h,t);
  else
    return new PlainDogBot(nm,h,t);
}
```

- Factory methods are normal methods which work like constructors, they return new objects 工厂方法是像构造函数一样工作的普通方法，它们返回新对象
- Constructor calls are preceded by the word new, factory methods are called like any other method 构造函数调用前要加上new这个词，而工厂方法的调用方式与其他方法一样
- Unlike constructors, factory methods can have an interface type as their return type 与构造函数不同，工厂方法可以将接口类型作为其返回类型
- They are used when we want to determine the actual type of a new object at run time 当我们想要在运行时确定新对象的实际类型时，使用它们
- They can be used to hide the actual type of the new object 它们可以用来隐藏新对象的实际类型

Factory Methods for Wrapping Objects

A factory method could produce a new wrapped object, or an object which puts a wrapper around an existing object

工厂方法可以生成新的包装对象，也可以生成在现有对象周围放置包装器的对象

```
static Quackable makeCountedDuck(String name){
  Duck wrappedDuck = new Duck(name);
  return new GloballyCountedQuacker(wrappedDuck);
}
```

- GloballyCountedQuacker就是很常规的wrapper

```
class GloballyCountedQuacker implements Quackable
{
 private Quackable  myQuacker;
 private static int count=0;

 public GloballyCountedQuacker(Quackable q)
 {
  myQuacker=q;
 }

 public int getCount()
 {
   return count;
 }

 public void quack()
 {
  count++;
  myQuacker.quack();
 }
}
```

- The difference between GloballyCountedQuacker and the previous QuackCounter is that here there is just one count for all the calls of the method quack() on all the Quackable objects returned
- GloballyCountedQuacker与之前的QuackCounter之间的区别在于，这里对所有返回的Quackable对象的方法quack()的所有调用只有一个计数

Factory objects

Factory methods may be static methods, or they may be called on objects

工厂方法可以是静态方法，也可以在对象上调用

An object which is designed to have factory methods called on it is called a **Factory Object**

被设计为在其上调用工厂方法的对象称为工厂对象

A factory object is used when we want to delegate the construction of new objects

当我们想要委托新对象的构造时，使用工厂对象

Factory object enable us to write generalised code whose effects depend on the particular factory object passed into it

工厂对象使我们能够编写通用代码，其效果取决于传递给它的特定工厂对象

Some of the aspects of the objects created by a factory method may be stored in the factory object

由工厂方法创建的对象的某些方面可能存储在工厂对象中

A factory object may be used to hide the real type of the objects it returns

工厂对象可以用来隐藏它返回的对象的真实类型

举个例子：

```
interface DogFactory {
  public DogBot makeDogBot(String name);
```

```
}
class PlainDogFactory implements DogFactory {
  private int hungry,tired;
  public PlainDogFactory(int h, int t) {
    hungry=h;
    tired=t;
  }
  public DogBot makeDogBot(String name) {
    return new PlainDogBot(name,hungry,tired);
  }
}
```

- This is a class of objects which make *DogBot*s with fixed initial hungry and tired values 这是一类使DogBot具有固定初始饥饿和疲劳值的对象

```
DogFactory factory = new PlainDogFactory(4,2);
DogBot dog1 = factory.makeDogBot("Patch");
DogBot dog2 = factory.makeDogBot("Rover");
```

Factory objects和observer结合

```
class SpyDogFactory extends PlainDogFactory {
  private Observer spy;
  public SpyDogFactory(int h, int t, Observer obs) {
    super(h,t);
    spy=obs;
  }
  public DogBot makeDogBot(String name) {
    DogBot d1 = super.makeDog(name);
    DogBot d2 = new ObservableDogBot(d1);
    d2.addObserver(spy);
    return d2;
  }
}
```

- This is a class of objects which make *DogBot*s which are all observed by an Observer object set when the SpyDogFactory object is created 这是一个类对象，当SpyDogFactory对象被创建时，这些DogBot对象都被一个观察者对象所观察到

```
Observer boss = new DogWatcher("Matthew");
DogFactory factory = new SpyDogFactory(4,2,boss);
```

```
DogBot dog1 = factory.makeDogBot("Patch");
DogBot dog2 = factory.makeDogBot("Rover");
```

The Singleton Design Pattern 单例设计模式

Another reason for using factory methods is that a constructor must always return a new object, but factory methods can return an existing object instead of a new object.

使用工厂方法的另一个原因是，构造函数必须始终返回新对象，但工厂方法可以返回现有对象而不是新对象。

【关于这里，我查了一下单例设计模式是什么，和PPT这里完全不一样，所以不清楚PPT这里说的是什么“singleton”，有知道的朋友可以跟我说。对比分析下来，唯一的不同就是在makeDogBot这里返回了类里的对象theOnlyDogBot】

```
class SingleDogFactory implements DogFactory
{
 private int hungry,tired;
 private DogBot theOnlyDogBot;

 public SingleDogFactory(int h,int t,String name)
 {
  theOnlyDogBot = new PlainDogBot(name,h,t);
 }

 public DogBot makeDogBot(String name)
 {
  return theOnlyDogBot;
 }
}
```

但可能会导致aliasing，就用另一个类来隐藏

- Here is a sneaky way to hide the aliasing:

```
class AliasDogBot implements DogBot {
 private String name;
 private DogBot realDogBot;

 public AliasDogBot(String nm, DogBot dog) {
  name=nm;
  realDogBot=dog;
 }

 public String getName() {
  return name;
 }

 public boolean eat() {
  return realDogBot.eat();
 }

 public void rest(){
  realDogBot.rest();
 }

 public void play() {
  realDogBot.play();
 }

 public String noise() {
  return realDogBot.noise();
 }
```

Replace
return theOnlyDogBot in SingleDogFactory
by

```
return new
AliasDogBot(name,theOnlyDogBot);
```

The Object Pool Design Pattern 对象池设计模式

Object Pool (also called Intern), like Singleton, can returns an object that was created before

对象池(也称为Intern)与Singleton一样，可以返回以前创建的对象

However, instead of one object it keeps a list of objects

但是，它保存的不是一个对象，而是一个对象列表

It is used when the objects returned are immutable

当返回的对象是不可变的时使用

```
class ColourMaker {
  private Map<Colour,Colour> colours;

  public ColourMaker {
   colours = new TreeMap<Colour,Colour>();
  }

  public Colour makeColour(int red, int green, int blue){
   Colour col1 = new Colour(red,green,blue);
   Colour col2 = colours.get(col1);
   if(col2==null) {
      colours.put(col1,col1);
      return col1;
   }
   else return col2;
  }
 }
```

算是singletom的复合版

The Strategy Design Pattern 策略设计模式

Suppose we want to call the various operations possible on a List of DogBots. Instead of writing a separate method for each we can generalise by using the idea of a function object, which is an object whose only job is to carry out an action.

假设我们想调用一个DogBots列表上可能的各种操作。我们可以使用函数对象的概念进行概括，而不是为每个对象编写单独的方法，函数对象是一个唯一的任务是执行操作的对象。

- Here is the interface type for such an object:

```
interface DogAction {
 public void action(DogBot d);
}
```

- Here is the generalised code that uses it:

```
void doIt(List<? extends DogBot> list, DogAction act)
{
 for(DogBot dog : list)
    act.action(dog)
}
```

代码很好理解，就是概括了各种操作，一种泛化

Generalising code by turning an action into a parameter is called the Strategy Design Pattern

通过将操作转换为参数来泛化代码称为策略设计模式

具体化一点：

Here is the class for the function object for calling `play` on a `DogBot`:

```
class MakePlay implements DogAction
{
  public void action(DogBot d)
  {
   d.play();
  }
}
```

Here is the class for the function object for calling `eat` on a `DogBot`:

```
class MakeEat implements DogAction
{
  public void action(DogBot d)
  {
   d.eat();  // Ignores the return value
  }
}
```

Here is a call which results in `play` being called on every object in the list `dogList`:

```
doIt(dogList, new MakePlay())
```

其他例子看PPT Topic16 P51–52

The State Design Pattern 状态设计模式

The State design pattern is used when we want to change the way we represent a type of object dynamically

当我们想要动态地改变表示对象类型的方式时，使用状态设计模式

Consider the idea of a set, it is a collection in which each possible element is either in the collection or it is not, there is no concept of elements occurring multiple numbers of times or having a position in the collection.

考虑集合的概念，它是一个集合，其中每个可能的元素要么在集合中，要么不在集合中，不存在元素出现多次或在集合中有位置的概念。

PPT里的例子有点难理解，但十分建议看懂，P53–56。

为了理解状态设计，我用个别的例子（来源：设计模式（Design Patterns） – 黄文博 – 博客园 (cnblogs.com)）

> 核心思想就是：当对象的状态改变时，同时改变其行为，很好理解！就拿QQ来说，有几种状态，在线、隐身、忙碌等，每个状态对应不同的操作，而且你的好友也能看到你的状态，所以，状态模式就两点：1、可以通过改变状态来获得不同的行为。2、你的好友能同时看到你的变化。
>
> State类是个状态类，Context类可以实现切换，我们来看看代码：

```
//核心类
public class State {


    private String value;


    public String getValue() {
        return value;
    }
```

```
    public void setValue(String value) {
        this.value = value;
    }

    public void method1(){
        System.out.println("execute the first opt!");
    }

    public void method2(){
        System.out.println("execute the second opt!");
    }
}
//切换类
public class Context {

    private State state;

    public Context(State state) {
        this.state = state;
    }

    public State getState() {
        return state;
    }

    public void setState(State state) {
        this.state = state;
    }

    public void method() {
        if (state.getValue().equals("state1")) {
            state.method1();
        } else if (state.getValue().equals("state2")) {
            state.method2();
        }
    }
```

```
}
```

The Bridge Design Pattern 桥接模式

桥接模式就是把事物和其具体实现分开，使他们可以各自独立的变化。桥接的用意是：将抽象化与实现化解耦，使得二者可以独立变化

【同样，PPT的例子还是很难理解，但还是建议去看看，Topic15 P57-58】

继续看代码：

【来源：设计模式（Design Patterns） - 黄文博 - 博客园 (cnblogs.com)】

```
public interface Sourceable {
  public void method();
} //先定义接口
```

- 分别定义两个实现类：

```
public class SourceSub1 implements Sourceable {

    @Override
    public void method() {
        System.out.println("this is the first sub!");
    }
}
public class SourceSub2 implements Sourceable {

    @Override
    public void method() {
        System.out.println("this is the second sub!");
    }
}
```

- 定义一个桥，持有Sourceable的一个实例：

```
public abstract class Bridge {
    private Sourceable source;

    public void method(){
        source.method();
    }

    public Sourceable getSource() {
```

```
        return source;
    }

    public void setSource(Sourceable source) {
        this.source = source;
    }
}
public class MyBridge extends Bridge {
  public void method(){
    getSource().method();
  }
}
```

- 这样，就通过对Bridge类的调用，实现了对接口Sourceable的实现类SourceSub1和SourceSub2的调用。

The Flyweight Pattern 享元模式

The Flyweight design pattern is where objects have an immutable part which can be shared.

在Flyweight设计模式中，对象具有可共享的不可变部分。

The ColouredRectangle example we saw previously would be an example of the Flyweight pattern if the Colour part of a ColouredRectangle was produced using a ColourMaker object as given previously:

我们之前看到的colourredrectangle的例子是Flyweight模式的一个例子，如果colourredrectangle的Colour部分是使用前面给出的colourmaker对象生成的:

```
class ColouredRectangle extends PlainRectangle {
  private static ColourMaker maker = new ColourMaker();
  private Colour colour;
  public ColouredRectangle(int h, int w, int red, int green, int blue){
    super(h,w);
    colour=maker.makeColour(red,blue,green);//colour部分是由colourmaker生成的，不可变
  }
  public Colour getColour() {
    return colour;
  }
  public void setColour(Colour c) {
    colour=c;
  }
}
```

Design pattern到这里结束，接下来是一些偏理论和记忆的东西了

Free Software

The word “free” in English has two meanings:

- Available without requirement of payment 无需付款即可使用
- Able to do whatever you want 可以做任何你想做的事

Conventional software is produced to meet a customer need, either it is produced for an individual customer who has asked for it, or it is built for open sale to customers who see it advertised and purchase it

传统的软件是为满足客户需求而生产的，要么是为要求它的个人客户生产的，要么是为看到广告并购买它的客户公开销售而生产的

Those who produce conventional software expect to get paid for it

那些制作传统软件的人希望从中获得报酬

- In order to protect their income, they will impose restrictions on how it can be used, which will usually mean it cannot be passed on to anyone else, and that it cannot be modified in unauthorised ways 为了保护他们的收入，他们会对数据的使用方式施加限制，这通常意味着数据不能传递给其他人，也不能以未经授权的方式进行修改
- As part of this protection, usually the source code is not made available, what is distributed is the machine executable code produced by compilers, which is difficult for humans to understand and modify 作为这种保护的一部分，通常源代码是不可用的，分发的是由编译器产生的机器可执行代码，这是人类难以理解和修改的

The phrase “open source” derives from the idea of the source code being publicly available, but the phrase “Open Source Software”(OSS) is usually used to mean software which is free of charge, and free of legal restrictions on how it can be used

“开源”一词源于源代码是公开可用的想法，但“开源软件”(OSS)一词通常用来表示免费的软件，并且在如何使用它方面没有法律限制

Why work for free?

Many people enjoy computer programming so much, they would do it as a hobby even if it wasn’t their job

许多人非常喜欢计算机编程，即使这不是他们的工作，他们也会把它作为一种爱好

Early OSS was sometimes hobby code

早期的OSS有时是业余代码

However, many complex software products in day-to-day use are OSS, it is not just a small-scale thing

然而，许多复杂的软件产品在日常使用中都是OSS，它不仅仅是一个小规模的事情

Many of the early developers of OSS were people who had strong beliefs about personal freedom, and saw distributing their free software as a way of giving people freedom instead of being dependent on commercial companies

许多OSS的早期开发人员都是对个人自由有强烈信念的人，他们认为发布他们的自由软件是给人们自由的一种方式，而不是依赖于商业公司

However, the success of OSS products has led many to support its use just because they see it as providing good reliable products

然而，OSS产品的成功已经导致许多人支持它的使用，因为他们认为它提供了良好可靠的产品

OSS is particularly common for software systems which are used to support software development, such as the Linux operating system and the Eclipse development environment, the Spring application framework

OSS特别适用于用于支持软件开发的软件系统，例如Linux操作系统和Eclipse开发环境、Spring应用程序框架

In some of these cases it is software produced for personal use that was spread more widely because it was found to be effective

在某些情况下，为个人使用而制作的软件因为被发现有效而得到了更广泛的传播

OSS Business Strategies

Some who produce OSS may be motivated entirely through altruistic reasons

一些开发OSS的人可能完全是出于利他主义的原因

Some may like the fame that comes from producing a successful and widely used product, even if it doesn't come with a fortune

有些人可能喜欢生产一款成功且广泛使用的产品所带来的名声，即使它不会带来财富.

However, others may see OSS as a "loss leader" (in business terms, a product sold at a low price to stimulate sales of more profitable goods or services)

然而，其他人可能将OSS视为"亏本的领导者"(在商业术语中，以低价销售的产品以刺激更有利可图的商品或服务的销售)。

There are companies which make profit from installation of OSS products, from offering training in the use of OSS products, and from providing consultancy advice to businesses about their use.

有些公司通过安装OSS产品、提供使用OSS产品的培训以及向企业提供有关使用OSS产品的咨询意见来获利。

- One example is Red Hat, which now has an annual revenue of over one billion US dollars. See: http://www.redhat.com/en/open-source

The Java programming language has many of the aspects of OSS, although it does not fully fit the definition because modification and redistribution of Java's support code is banned by its developers (Sun Microsystems, now owned by Oracle Corporation)

Java编程语言具有OSS的许多方面，尽管它并不完全符合定义，因为Java的开发人员(Sun Microsystems，现在归Oracle公司所有)禁止修改和重新分发Java的支持代码。

One of the reasons for the success of Java was because it was distributed free of charge

Java成功的原因之一是它是免费发布的

OSS Development Methodology

Software produced by volunteers cannot be produced under the forms of management used for commercial software

志愿者制作的软件不能采用商业软件的管理形式

Large scale OSS systems have to be produced by teams, like any other large scale software products

像任何其他大型软件产品一样，大型OSS系统必须由团队生产

There needs to be an overall co-ordinator, but assignment of individual tasks is done through volunteering

需要有一个整体协调员，但个人任务的分配是通过志愿者来完成的

A large scale OSS system will be developed by people working across the world, communicating electronically

一个大规模的OSS系统将由世界各地工作的人们开发出来，通过电子方式进行交流

It has some of the aspects of Agile software development, but the Agile methodology emphasises close personal interaction

它具有敏捷软件开发的一些方面，但是敏捷方法论强调密切的个人交互

With OSS systems there is typically no developer–customer distinction, the developers are also customers as they will make use of the system themselves

对于OSS系统，通常没有开发人员–客户的区别，开发人员也是客户，因为他们自己将使用系统

One of the pioneers of OSS, Eric S. Raymond, described the distinction between development of software by a central business organisation and by a community of volunteers as like that between “The Cathedral and the Bazaar”

OSS的先驱之一Eric S. Raymond将中央商业组织和志愿者社区开发软件的区别描述为“大教堂和集市”之间的区别。

- A cathedral is a large building built to a single plan under close control, which typically takes a long time to build 大教堂是在严密控制下按照单一平面图建造的大型建筑，通常需要很长时间才能建成
- A bazaar is an informal collection of stalls, which can change quickly and seems to have no central control, relying on stallholders co–operating, but each working to their own goals 集市是一个非正式的摊位集合，它可以快速变化，似乎没有中央控制，依靠摊主的合作，但每个人都为自己的目标而努力
- Commercial software, and early OSS systems were developed like a cathedral 商业软件和早期的OSS系统的发展就像一座大教堂
- Development as a bazaar means customers act as testers and debuggers, good quality stalls stay, bad quality stalls go through lack of business or are improved through customer suggestion 作为集市的开发意味着客户充当测试者和调试者，质量好的摊位留下来，质量差的摊位因缺乏业务而被淘汰或通过客户建议得到改进
- A bazaar may have a co–ordinator who manages the recruitment of stall–holders, but does not have a strong overall plan and leaves the stall–holders free to work as they want 集市可能会有一个协调者负责管理摊贩的招聘，但没有一个强有力的整体计划，让摊贩自由地按照自己的意愿工作

Many Eyeballs Tame Complexity

In OSS systems, the developers are typically also users of the software

在OSS系统中，开发人员通常也是软件的用户

Users who are also developers can report bugs if they find them and suggest corrections based on being able to see the source code

同时也是开发人员的用户可以在发现错误后报告错误，并根据能够看到源代码的情况提出更正建议

The result is that there are a large number of users who also act as testers

其结果是有大量用户同时充当测试人员

As the software is source code rather than compiled machine executable code, it can easily be updated to correct a bug, it does not have to wait until a formal release of a new version

由于该软件是源代码而不是编译后的机器可执行代码，因此可以很容易地对其进行更新以纠正错误，而不必等到新版本的正式发布

Typically, a large OSS system will be divided into many small parts, with correction of a bug in one part not having an impact on development of other parts

通常，一个大型的OSS系统会被分成许多小的部分，其中一个部分的错误纠正不会对其他部分的开发产生影响

Open Source and Business Risk

If you run a company which needs a software product, you could develop your own, but this would be expensive to do and to maintain as your requirements change

如果你经营一家需要软件产品的公司，你可以开发自己的软件产品，但这将是昂贵的，并且随着需求的变化而维护

You could purchase a product from a closed-source supplier, but then you are tied to that supplier: you are forced to carry on making use of them and paying what they ask for further development, because only they have knowledge and control of the code

您可以从闭源供应商处购买产品，但随后您就被捆绑在该供应商身上:您被迫继续使用他们，并支付他们要求的进一步开发费用，因为只有他们拥有知识并控制代码

If you use an OSS product, you have the source code, and there is likely to be several service companies who will help adapt it to your needs

如果您使用的是OSS产品，那么您就拥有了源代码，并且很可能有几个服务公司将帮助您调整它以满足您的需求

So using an OSS product means there is no risk that a company supplying the software to you goes out of business, or makes unreasonable charges because it knows you have no choice but to carry on using it

因此，使用OSS产品意味着不存在向您提供软件的公司破产的风险，或者因为知道您别无选择，只能继续使用它而收取不合理的费用

Software Freedom

The strongest form of “free software” insists on the following four freedoms:

最强大形式的“自由软件”坚持以下四个自由:

- The freedom to run the program, for any purpose 出于任何目的而运行程序的自由
- The freedom to study how the program works, and change it so it does your computing as you wish. Access to the source code is a precondition for this. 研究程序如何工作的自由，并改变它，使其按照您的意愿进行计算。访问源代码是实现这一目标的先决条件。
- The freedom to redistribute copies so you can help your neighbour. 重新发布副本的自由，这样你就可以帮助你的邻居。
- The freedom to distribute copies of your modified versions to others. By doing this you can give the whole community a chance to benefit from your changes. Access to the source code is a precondition for this. 将修改后的版本分发给他人的自由。通过这样做，您可以让整个社区有机会从您的更改中受益。访问源代码是实现这一目标的先决条件

Copyleft 非盈利版权

The supporters of Free Software have developed a concept they call copyleft to mean using the copyright laws in a way that guarantees the freedom of others to copy and use their work rather than stopping unauthorised or unpaid copying

自由软件的支持者已经发展了一个概念，他们称之为copyleft，意思是使用版权法来保证其他人复制和使用他们作品的自由，而不是阻止未经授权或无偿的复制

It involves a free software license, which is associated with free software, use of that software is granted only to

those who agree to the terms of the free software license

它涉及自由软件许可证，它与自由软件相关联，该软件的使用只授予那些同意自由软件许可证条款的人

The terms will generally state that users of the software may pass it on to others only if when it is passed on, those it is passed to also agree to the terms of the same license

这些条款通常会声明，只有当软件被传递给其他人时，被传递的人也同意同一许可证的条款，软件的用户才可以将其传递给其他人

The license will include the requirement that the source code is made available, and also state the extent to which the code may be modified and how the modifications should be acknowledged

许可证将包括要求源代码是可用的，并且还说明代码可以修改的程度以及如何承认修改

There are many variations on this form of license, the most commonly used form is the GNU General Public License, developed by Richard Stallman

这种形式的许可证有许多变体，最常用的形式是由Richard Stallman开发的GNU通用公共许可证

If the terms of the license do not insist that copies and adaptions are passed on with the same license, it is not copyleft

如果许可协议的条款没有坚持副本和改编版本以相同的许可协议传递，那么它就不是copyleft

Core and Community

In successful Open Source projects, there will generally be a small core of people (10–15) who are active in controlling the code base, and deciding on new functionality

在成功的开源项目中，通常会有一小部分核心人员(10–15人)积极地控制代码库，并决定新的功能

A larger group of people will have some involvement in correcting defects in the software

更大的一组人将参与到纠正软件中的缺陷中

A larger group than this will be actively involved in reporting problems with the software

一个比这更大的小组将积极参与报告软件的问题

Volunteers may submit code to implement proposed new features, the project may benefit from picking the best from several implementations

志愿者可以提交代码来实现提议的新功能，项目可以从几个实现中选择最好的

This system works best when, as with GNU/Linux, the overall product naturally divides into many semi–independent parts

当整个产品自然地分成许多半独立的部分时，这个系统工作得最好，就像GNU/Linux一样

A fork is when there is a split, and a separate group starts maintaining a separate version of the product

分叉是指有一个分裂，一个单独的小组开始维护产品的单独版本

In most cases, developers will prefer to contribute to an established product rather than fork it and develop their own new version

在大多数情况下，开发人员更愿意为已建立的产品做出贡献，而不是分叉它并开发自己的新版本

So enabling volunteers to contribute to an OSS product helps maintain one standard product rather than seeing it split into many versions

因此，允许志愿者为OSS产品做出贡献有助于维护一个标准产品，而不是看到它分裂成许多版本

Open Source Governance 开源治理

Proprietary software is owned by the organisation which produces it, and controlled by a business management structure

专有软件由生产它的组织拥有，并由业务管理结构控制

Open Source software development generally has a control structure which means an identifiable official version of the product exists

开源软件开发通常有一个控制结构，这意味着产品存在一个可识别的官方版本

In some cases, this is a “benevolent dictatorship”, meaning the person who first proposed the project always retains the final say, examples include Linus Torvalds with Linux and Guido van Rossum with the Python programming language

在某些情况下，这是一种“仁慈的独裁”，意味着最初提出项目的人总是拥有最终决定权，例如Linux的Linus Torvalds和Python编程语言的Guido van Rossum

In others, there may be an committee structure involving some form of election, an example is the Apache Software Foundation, which is responsible for the Tomcat Java servlet system, the Struts Java web application framework, and many other projects

在其他情况下，可能会有一个委员会结构，涉及某种形式的选举，例如Apache软件基金会，它负责Tomcat Java servlet系统、Struts Java web应用程序框架和许多其他项目

The mobile device operating system Android is open-source, but controlled by the private company Google

移动设备操作系统Android是开源的，但由私人公司谷歌控制

The OSS nature of Android makes it easy to write applications for it, and has led to its widespread adoption, but Google have kept control by working internally on each new version, publishing the source code only when the version is released

Android的OSS特性使得为其编写应用程序变得很容易，并导致了它的广泛采用，但谷歌通过内部工作来控制每个新版本，只有在版本发布时才发布源代码

Open Source Software: Summary

Open Source Software (OSS), has developed a long way from the time when it was seen as a few eccentrics putting together code as a hobby

开源软件(OSS)，从被看作是几个怪人把代码放在一起作为业余爱好的时候，已经发展了很长一段路

Most standard software development tools are now OSS, and OSS is moving into other areas

大多数标准软件开发工具现在都是OSS，而且OSS正在进入其他领域

The key to its success is that the community of users of the code also provides a community of testers and debuggers

它成功的关键在于代码的用户社区也提供了一个测试人员和调试器社区

OSS develops rapidly in requirement to need because anyone who wants to improve it can do so

OSS在需求中迅速发展，因为任何人想要改进它都可以这样做

OSS products avoid the risk of control of code that is vital for a business being in another business’s hands

OSS产品避免了代码控制的风险，这对于一个企业掌握在另一个企业手中是至关重要的

OSS products maintain a single identity because it is in the interests of their users and developers to contribute

to one standard product rather than “fork”

OSS产品保持单一的身份，因为为一个标准产品而不是“分支”做出贡献符合用户和开发人员的利益。

OSS products maintain their OSS nature through a use of copyright legislation to enforce free distribution rather than prevent it

开源软件产品通过使用版权立法来加强自由发布而不是阻止它，从而保持其开源软件的本质

Control of the development of OSS products (“governance”) may now be with companies operating on a commercial basis rather than individuals whose motivation is enjoyment of software and/or personal freedom

对OSS产品开发的控制(“治理”)现在可能由基于商业基础的公司而不是以享受软件和/或个人自由为动机的个人来控制

Microsoft’s Best Practices

The following is a list of “best practices” for software development projects which was put together by Microsoft:

- Revision Control System 修订控制系统
- Daily Builds 每日构建
- Build Verification Tests 创建确认测试 / 识别小版本验证测试用例
- Bug Database 缺陷库 / 缺陷跟踪管理系统
- Code Reviews 代码复查
- Coding and Engineering Guidelines 编码及工程指引
- Code Analysis Tools 代码分析工具
- Globalization and Localization 全球化和本土化

This is about adopting a disciplined approach to managing the code files andinformation of a large scale project. There will often be dedicated Computer Aided Software Engineering (CASE) tools for managing these practice

这是关于采用一种有纪律的方法来管理大型项目的代码文件和信息。通常会有专门的计算机辅助软件工程(CASE)工具来管理这些实践

This lecture is based on Microsoft’s description of these best practices, with some additional comments noting further developments which have been promoted by the Extreme Programming movement

这堂课是基于微软对这些最佳实践的描述，加上一些额外的评论，指出了极限编程运动所推动的进一步发展

Revision Control System

A Revision Control System (also called “Version Control”) keeps a record of all the changes that are made to the software of a project

修订控制系统(也称为“版本控制”)保存对项目软件所做的所有更改的记录

- It will generally incorporate **roll-back** features, enabling a return to any previous version of the system 它通常包含回滚功能，允许返回到系统的任何先前版本
- A **check-out** is when a developer obtains the current version of a file, to have a local copy to work with 签出是指开发人员获取文件的当前版本，以便有一个本地副本可以使用
- A **check-in** (or “commit”) is when a developer enters a modified version of the file into the system to

become the current version (“master copy”) of that file for the system 签入(或“提交”)是指开发人员将文件的修改版本输入到系统中，以成为该文件的当前版本(“主副本”)

- A **conflict** is when two different developers are trying to make incompatible changes to the same file 冲突是指两个不同的开发人员试图对同一个文件进行不兼容的更改
- A **merge** is an attempt to put together separate changes made by two or more developers 合并是将两个或多个开发人员所做的单独更改放在一起的尝试

Conflicts could be avoided by allowing only one developer to work on any one file at one time (“locking” the file), but this is not always practical or efficient in large scale systems

可以通过一次只允许一个开发人员处理任何一个文件(“锁定”文件)来避免冲突，但这在大规模系统中并不总是实用或有效的

Daily Build

A build is the process of producing a complete working version of the software for the whole system, compiling the source code files and linking them

构建是为整个系统生成软件的完整工作版本，编译源代码文件并将它们链接起来的过程

For all but very small systems this can become complex, so it is good practice to use dedicated tools such for this, which will work with build files that give instruction for the build of a system

对于除了非常小的系统之外的所有系统，这可能会变得复杂，因此使用专门的工具是一个很好的实践，它将与构建文件一起工作，为系统的构建提供指导

Having a daily build means having a practice of doing this once a day, usually overnight so the team starts work with the latest version of the software

进行每日构建意味着每天都要这样做一次，通常是在晚上，这样团队就可以开始使用软件的最新版本

A build will generally contain at least some verification tests for correct working, sometimes referred to as “smoke tests”

构建通常至少包含一些正确工作的验证测试，有时称为“冒烟测试”。

A build break is when the build or its tests fail, it should be considered a high priority problem that must be fixed immediately

构建中断是当构建或其测试失败时，应该将其视为必须立即修复的高优先级问题

Continuous Integration

The Extreme Programming movement recommends a complete build of the system for every check-in of a modified file

极限编程运动建议在每次签入修改的文件时对系统进行完整的构建

Extreme Programming also recommends test-driven development, so there is a full range of tests for every aspect of the system, and regression testing meaning the build incorporates all tests

极限编程还推荐测试驱动开发，因此系统的每个方面都有完整的测试范围，回归测试意味着构建包含所有测试

This practice of builds involving full testing for every modification is called continuous integration

这种对每个修改进行全面测试的构建实践被称为持续集成

Developers will have their own copies of the system code for making and testing modifications, but cannot check-in their modifications unless they satisfy the tests in the build

开发人员将拥有他们自己的系统代码副本，用于制作和测试修改，但是除非他们满足构建中的测试，否则不能签入他们的修改

With continuous integration it is recommended that developers check-in their work at least once a day

对于持续集成，建议开发人员每天至少签入一次他们的工作

For this work pattern to be possible, automated build tools which incorporate testing are essential

为了使这种工作模式成为可能，包含测试的自动化构建工具是必不可少的

A ten-minute build (build script that completes all building including testing in ten minutes) is recommended

推荐一个10分钟的构建(在10分钟内完成包括测试在内的所有构建的构建脚本)

Version Control and Build tools

- Commonly used version control tools include:
  - Subversion http://subversion.apache.org
  - Git http://git-scm.com
  - Helix Core https://www.perforce.com/products/helix-core
  - Mercurial https://www.mercurial-scm.org
- Commonly used build tools include:
  - Apache Ant http://ant.apache.org
  - Apache Maven http://maven.apache.org
  - GNU Make https://www.gnu.org/software/make
- Note most of these are Open Source Software

Assert statements

An assert statement in source code is a statement involving an expression which is always meant to evaluate to true, if it does not an exception is thrown

源代码中的assert语句是一个涉及表达式的语句，该表达式的求值总是意味着为真，如果不为真则抛出异常

Assert statements are used for testing software during development, compilers will have an option to ignore them when producing compiled code for release

Assert语句在开发过程中用于测试软件，编译器在生成编译后的代码发布时可以选择忽略它们

Java has had assert statements since version 1.4

Java从1.4版开始就有了断言语句

For example, suppose you had a variable of type int called age which you expected always to be set to a non-negative number, then you could put the following in your program:

例如，假设你有一个int类型的变量age，你希望它总是被设置为一个非负数，那么你可以在你的程序中输入以下语句:

```
assert age>=0;
```

It would alert you to any case where that variable was wrongly set to a negative number, which might not otherwise be spotted until it caused an error in output somewhere else

它会提醒您该变量被错误地设置为负数的任何情况，否则可能不会被发现，直到它在其他地方的输出中导致错误

Unit testing

Extreme Programming recommends built-in testing incorporated into all elements of the code, this is called unit testing

极限编程建议将内置测试整合到代码的所有元素中，这被称为单元测试

JUnit is a commonly used framework (set of classes) which supports the development of automated tests using the idea of assertion, see http://junit.org

JUnit是一种常用的框架(类集)，它支持使用断言思想开发自动化测试，请参阅http://junit.org

A unit test is meant to test an individual class or method, it should run very quickly in order to enable a "ten-minute build"

单元测试意味着测试单个类或方法，它应该运行得非常快，以便实现"十分钟构建"。

Unit tests may make use of mock objects, which are simple objects that replace the rest of the code that a objects of a class would interact with when it is incorporated into the whole system

单元测试可以使用模拟对象，这些对象是简单的对象，当类的对象被合并到整个系统中时，它们将取代与之交互的代码的其余部分

The idea is that the extra work involved in developing automated tests is repaid by the way they ensure modifications made to the code cannot have unexpected effects on other parts of the code, because that would be shown up by an assertion test failing in the build

其思想是，开发自动化测试所涉及的额外工作通过确保对代码所做的修改不会对代码的其他部分产生意外影响的方式得到回报，因为这将在构建中的断言测试失败中显示出来

Bug Database

A bug database is a central record for bugs that have been identified in the system

bug数据库是系统中已识别的bug的中央记录

The record for a particular bug will describe it, give tests that demonstrate it, give its status (active, resolved or closed), and who has been assigned to correct it

特定错误的记录将描述它，给出证明它的测试，给出它的状态(活动的、解决的或关闭的)，以及分配给谁来纠正它

Tests that demonstrate a bug should say what is needed to get the bug to occur, what the system should do if there was no bug, and what is observed instead because of the bug

演示bug的测试应该说明导致bug发生需要做什么，如果没有bug系统应该做什么，以及由于bug而观察到什么

A bug should also be given a severity and priority rating

还应该给bug指定严重性和优先级

- An active bug is one which has been reported but not corrected 一个活跃的bug是一个已经被报告但没有被纠正的bug
- A resolved bug is one where a programmer has submitted a correction 已解决的bug是程序员提交了更正的bug
- A closed bug is where the person who originally reported the bug agrees that the correction made has removed it 关闭的bug是指最初报告错误的人同意所做的更正已经删除了它

War Team and Bug Triage

Microsoft recommends the technique of a war team, which is a small team of senior managers who take control of a project when it is near completion

微软建议采用作战团队的技术，即由高级管理人员组成的小团队，在项目接近完成时接管项目

The war team has the task of ensuring the system is “good enough” for release

作战团队的任务是确保系统“足够好”，可以发布

The war team will triage remaining bugs, meaning they analyse them and decide how serious they are

作战小组将对剩余的漏洞进行分类，这意味着他们会分析它们并决定它们的严重程度

- Some reported bugs may be duplicates of others 一些报告的错误可能是其他错误的重复
- Some reported bugs may be “features” (agreed as acceptable according to the requirements of the system) 一些报告的bug可能是“特性”(根据系统的需求同意为可接受的)
- Some reported bugs may be low priority and can be left to be resolved in a future release 一些报告的bug可能是低优先级的，可以留到将来的版本中解决

The war team identify responsibility for fixing bugs which it is agreed cannot be allowed to be shipped in the next release

作战团队确定修复bug的责任，他们同意不能在下一个版本中发布

Code reviews and coding guidelines

A *code review* is a systematic examination of source code, it may involve a formal meeting with a structured format to an informal agreement for team members to check over each other’s code

代码审查是对源代码的系统检查，它可能包括一个正式的会议，一个结构化的格式，一个非正式的协议，让团队成员检查彼此的代码

Microsoft recommends code reviews because:

- “An extra pair of eyes can go a long way”“多一双眼睛会大有帮助”
- It “disciplines new hires very quickly on coding style”它“对新员工的编码风格进行快速训练”。

Note that the Extreme Programming practice of “pair programming” can be considered a continuous code review

请注意，极限编程的“结对编程”实践可以被视为连续的代码审查

There are software tools to help conduct and manage code reviews, enabling colleagues to view and comment on code

有一些软件工具可以帮助进行和管理代码审查，使同事能够查看和评论代码

Automated code review involves tools which analyse code to check for compliance with predefined sets of coding guideline rules or best practices

自动代码审查包括分析代码以检查是否符合预定义的编码指南规则或最佳实践的工具

Static and dynamic analysis tools

A **static analysis tool** is a tool which examines the source code without running it, for example:

静态分析工具是在不运行源代码的情况下检查源代码的工具，例如:

- Checkstyle — a tool to help programmers write Java code that adheres to a standard http://checkstyle.sourceforge.net

- FindBugs — a tool which uses static analysis to look for bugs in Java code http://findbugs.sourceforge.net

A **dynamic analysis tool** is a tool which examines what happens when source code is run

动态分析工具是一种检查源代码运行时发生的情况的工具

A **profiler** is a dynamic analysis tool which analyses the time and space usage of a program, in particular which parts of the program are most frequently used, or which parts are taking up the most memory — these are the parts where any code change to improve efficiency should be made, it is often hard to predict where they are without using such a tool. The VisualVM profiler for Java https://visualvm.github.io is an example

分析器是一种动态分析工具，它分析程序的时间和空间使用情况，特别是程序的哪些部分最常被使用，或者哪些部分占用了最多的内存——这些是应该进行任何代码更改以提高效率的部分，如果不使用这种工具，通常很难预测它们在哪里。Java的VisualVM分析器https://visualvm.github.io 就是一个例子

Debuggers

A debugger is a tool for assisting debugging

调试器是辅助调试的工具

There are a variety of types of debugging tools, but typically a debugger will enable a programmer to run a program and cause it to suspend at times in the execution, it will have features which enable the programmer to investigate the dynamic state of the program execution when it is suspended, looking at the value of variables and possibly changing them

有多种类型的调试工具，但典型的调试器将使程序员能够运行程序并使其在执行过程中暂停，它将具有使程序员能够在程序暂停时调查程序执行的动态状态，查看变量的值并可能更改它们的功能

A debugger may run code step-by-step (suspending after every execution step) or may enable the programmer to set breakpoints at which it suspends

调试器可以一步一步地运行代码(在每个执行步骤后暂停)，也可以允许程序员设置暂停的断点

A simple way of debugging is to put statements at various points in your program which print out the value of variables when they are reached, if the value of a particular variable goes from an expected one to an unexpected one between two print statements, you can removed the other print statements and put more in between these two

一种简单的调试方法是在程序的不同位置放置语句，当到达变量值时打印出它们的值，如果特定变量的值在两个打印语句之间从预期值变为意外值，您可以删除其他打印语句，并在这两个语句之间放置更多的语句

Systematic use of a debugger enables you to do the same thing without having to make changes to the code

系统地使用调试器使您可以在不更改代码的情况下执行相同的操作

9 essential debugging rules

These were developed by Dave Agans: http://gipsysoft.com/articles/debug-rules

- Understand the system — make sure you understand how any code you are re-using is meant to work 理解系统——确保你理解你重用的代码是如何工作的
- Make it fail — find an example which consistently shows the bug happening 让它失败——找到一个始终显示bug发生的例子
- Quit thinking and look — run the code using debugging tools to get information that will help you debug 停止思考，看看——使用调试工具运行代码，获取有助于调试的信息
- Divide and conquer — narrow your search until you reach the exact place where the bug occurs 分而治之

——缩小搜索范围，直到找到bug发生的确切位置

- Change just one thing at a time — change just one thing which you think causes the bug, if the bug still happens, change it back to what it was 一次只改变一件事——只改变你认为导致错误的一件事，如果错误仍然发生，就把它改回原来的样子
- Keep an audit — keep a record of the bug and what you have done to try to find and correct it 保持审计–保持一个记录的错误和你所做的努力，以找到和纠正它
- Check the obvious first — for example, is the code you are looking at the same code you are running? 首先检查最明显的——例如，你正在查看的代码和你正在运行的代码是一样的吗?
- Ask someone else — a fresh viewpoint can often help 问问别人——一个新的观点通常会有所帮助
- If you didn't fix it then it's not fixed — make sure any changes you made did actually correct the bug 如果你没有修复它，那么它就没有修复–确保你所做的任何更改都确实纠正了错误

Globalisation and Localisation

Globalisation is the process of making your software work in different languages and cultures

全球化是让你的软件在不同的语言和文化中工作的过程

Localisation is the actual translation work for individual languages

本地化是针对个别语言的实际翻译工作

The idea is so that your software product can easily be used by different people across the world

这个想法是为了让你的软件产品可以很容易地被世界各地不同的人使用

Software needs to be written in a way where aspects to do with display of text and other issues where they may be differences in different countries are separated out so they can easily be modified without any changes having to be made to the rest of the code

软件需要以一种方式编写，其中与文本显示和其他问题有关的方面在不同国家可能存在差异，因此可以轻松修改，而无需对其余代码进行任何更改

Issues include:

- Different languages 不同的语言
- Different scripts 不同的脚本
- Some languages read text right–to–left rather than left–to–right 有些语言从右到左读取文本，而不是从左到右
- Some languages have complex rules for combining characters 有些语言有复杂的字符组合规则
- Different formats for dates (UK and USA formats are different) 日期格式不同(英国和美国格式不同)
- Different paper sizes (standard paper size in USA is 279x216mm, standard paper size in most of the rest of the world is 297x210mm, known as "A4") 不同的纸张尺寸(美国的标准纸张尺寸为279x216mm，世界上大多数其他国家的标准纸张尺寸为297x210mm，称为"A4")

"Eat your own dog food"

The phrase "eat your own dog food" originated from the idea that if you ran a company that sold food for dogs, but you kept dogs and you did not feed those dogs on the dog food you sold, it suggests you think the dog food you sell is poor quality

“吃你自己的狗粮”这个短语起源于这样一个想法:如果你经营一家卖狗粮的公司，但你养了狗，而且你没有用你卖的狗粮喂那些狗，这表明你认为你卖的狗粮质量很差

In general it means if you are manufacturing and/or selling a product, but you do not use that product yourself, instead when you need something of that sort you use another product, it suggests you do not think your product is of high quality

一般来说，这意味着如果你正在制造和/或销售一种产品，但你自己不使用该产品，相反，当你需要那种产品时，你使用了另一种产品，这表明你认为你的产品质量不高

Making use of your own product also helps make sure you understand it, and an easily relate to any comments about it from your customers

利用你自己的产品也有助于确保你理解它，并且很容易与客户对它的任何评论联系起来

The phrase originated from television advertisements for a brand of dog food, but achieved widespread usage within Microsoft, and from there came to be used in general for software products, after an internal campaign within Microsoft to encourage the use of Windows and other Microsoft products when developing Microsoft software

这个短语起源于一个狗粮品牌的电视广告，但在微软内部得到了广泛的使用，在微软内部鼓励在开发微软软件时使用Windows和其他微软产品的运动之后，从那里开始一般用于软件产品